

Practica 9

Microcontroladores

Alumno: Itzel Estrada Arcos (368980)

Docente: Ing. Jesus Padron

Martes 29 de abril 2025

0.1 Introduction.

En esta tarea se abordó el estudio y la resolución de ecuaciones diferenciales ordinarias mediante distintos métodos simbólicos y numéricos, implementados en Python. A través de diversos códigos, se aplicaron técnicas clásicas como **Picard, Euler, Taylor, Runge-Kutta**, y métodos multietapa como **Adams-Bashforth**, así como herramientas de resolución modernas como `solve_ivp`. Estos enfoques me permitieron modelar y analizar distintos fenómenos reales, desde el crecimiento poblacional y el movimiento con resistencia hasta la dinámica de sistemas biológicos como el modelo depredador-presa de Lotka-Volterra.

0.2 Desarrollo de la practica.

Los códigos presentados exploran distintas formas de resolver ecuaciones diferenciales ordinarias, que son fundamentales para modelar fenómenos en la ciencia y la ingeniería. Se utilizan tanto herramientas simbólicas como métodos numéricos para aproximar soluciones. Los métodos aplicados van desde técnicas básicas hasta avanzadas, y se ilustran con ejemplos prácticos como el crecimiento poblacional, el movimiento de proyectiles, procesos químicos y modelos de interacción entre especies.

0.2.1 Proceso paso a paso.

Como se nos indico, en la tarea, preferí hacer un código por cada conjunto de ejercicios para que no fuera tan complicado en cuestion de resultados y mero desarrollo del código.

Código 5.1.

Primero, el código comienza importando la librería `SymPy`, la cual permite trabajar con álgebra simbólica en Python. Después, se definen las variables simbólicas necesarias: una variable independiente llamada t y una función $y(t)$, que es la incógnita de las ecuaciones diferenciales a resolver.

El código se enfoca en resolver cuatro ecuaciones diferenciales distintas, etiquetadas como incisos a, b, c y d. Para cada una de ellas, se construye una ecuación diferencial de primer orden de la forma $y'(t) = f(t, y)$. Estas ecuaciones se pasan a una función que resuelve simbólicamente la EDO utilizando el método `dsolve`, junto con una condición inicial que define un valor de y en un punto específico. Esto permite obtener una solución particular y exacta para cada caso.

Una vez obtenidas las soluciones, el código realiza un análisis para cada una de las funciones $f(t, y)$. Este análisis consiste en derivar f con respecto a y para comprobar si es continua y si cumple la condición de Lipschitz, una propiedad matemática importante que garantiza la existencia y unicidad de la solución. Este paso se hace calculando simbólicamente la derivada parcial f_y para cada inciso.

Después de resolver esos ejercicios, el código aborda un nuevo problema basado en el **método de Picard**. Este método es una técnica iterativa para aproximar la solución de una ecuación diferencial inicial cuando no se desea o no se puede obtener una solución exacta. Primero se define una función base $y(t)$, que en este caso es constante igual a 1. Luego, se construye la primera aproximación $y(t)$ integrando una función que depende de y .

Posteriormente, se generan nuevas aproximaciones y , y , y , repitiendo el mismo proceso, pero esta vez tomando como base la función anterior. Cada paso consiste en tomar la función anterior, reemplazarla en la expresión de la derivada, e integrar el resultado respecto a una variable auxiliar llamada t , desde 0 hasta t . Finalmente, todas estas aproximaciones se imprimen para mostrar cómo se va acercando la solución a medida que se repite el proceso.

Código 5.2.

Este programa está dividido en dos grandes secciones: la primera se enfoca en aplicar el método de Euler para aproximar soluciones de ecuaciones diferenciales ordinarias, y la segunda trata un modelo poblacional simbólico que se transforma para analizar la evolución de una proporción a lo largo del tiempo. El programa trabaja con diferentes ecuaciones diferenciales mediante el **método de Euler**. Para cada caso:

Se define el intervalo de tiempo desde un valor inicial hasta uno final, con un paso fijo. Luego se inicializa la variable dependiente y con un valor inicial.

En cada inciso, se especifica la forma de la ecuación diferencial y se itera sobre el número de pasos definidos. En cada iteración se calcula el valor de la derivada según la ecuación dada, se actualiza el valor de y utilizando la fórmula del método de Euler, y se incrementa el valor de t .

Después de cada iteración, se imprimen los valores actuales de t y y . Este proceso muestra cómo se va aproximando la solución paso a paso. Cada inciso trata una ecuación distinta, por lo que el término de derivada cambia, pero el procedimiento es el mismo.

En la segunda parte, correspondiente al Ejercicio 11, el enfoque cambia al análisis simbólico con **SymPy**. Aquí se trabaja con un sistema de ecuaciones diferenciales relacionadas con un modelo poblacional. Se definen funciones simbólicas para describir las tasas de cambio de dos poblaciones: una general y otra modificada. Se establece una proporción entre ambas poblaciones y se deriva la ecuación diferencial que describe cómo cambia esa proporción con el tiempo. Esta derivación se hace simbólicamente, permitiendo obtener una expresión simplificada.

Luego, en el inciso b, se usa el método de Euler para calcular una aproximación numérica de esa proporción a lo largo del tiempo. Se define el número de pasos, el valor inicial de la proporción y se aplica la fórmula iterativa para obtener los valores de cada paso. Al final se imprime el valor aproximado de la proporción luego de 50 unidades de tiempo.

En el inciso c, se obtiene la solución exacta de la ecuación de la proporción usando la fórmula general de la solución de EDOs separables. Se encuentra una

constante de integración que satisface la condición inicial y se sustituye para obtener la expresión explícita de la solución exacta. Se evalúa esta expresión en el tiempo 50 y se compara con el resultado del método de Euler.

Por último, se genera una gráfica donde se comparan las soluciones: la curva aproximada con Euler y la curva exacta obtenida simbólicamente. Esto permite visualizar la precisión del método numérico frente a la solución analítica.

Código 5.3.

Este programa se divide en dos partes principales. La primera aplica el **método de Taylor** de segundo orden para aproximar soluciones a ecuaciones diferenciales ordinarias, y la segunda analiza el **movimiento de un proyectil** con resistencia del aire.

En la primera parte, se define una función que implementa el método de Taylor de segundo orden. Para cada problema, se parte de una condición inicial y se calculan las derivadas necesarias para aplicar la fórmula de Taylor. En cada iteración se actualiza el valor de la función usando tanto la derivada como la derivada segunda aproximada. Se presentan los valores de t y y en cada paso para observar la evolución de la solución.

En la segunda parte, se resuelve el movimiento vertical de un proyectil teniendo en cuenta la resistencia del aire. Se considera una ecuación diferencial que describe cómo cambia la velocidad en función del tiempo, afectada por la gravedad y una fuerza de resistencia proporcional al cuadrado de la velocidad. En el inciso a, se aplica el método de Euler para calcular la velocidad del proyectil en pasos de 0.1 segundos hasta llegar a 1 segundo. En el inciso b, se determina el momento en que la velocidad se vuelve cero, lo que indica que el proyectil alcanzó su altura máxima.

En resumen, el código muestra cómo aplicar dos métodos numéricos distintos para resolver ecuaciones diferenciales: Taylor de segundo orden para trayectorias suaves con más precisión, y Euler para modelar dinámicas físicas como el movimiento con fricción del aire. También incluye una comparación entre la simulación numérica y el análisis físico del problema.

Código 5.4.

Primero, el código importa tres bibliotecas: **NumPy** para cálculos numéricos, **SymPy** para manipulación simbólica de funciones matemáticas, y **Matplotlib** para graficar. Estas herramientas permiten trabajar con ecuaciones diferenciales tanto en forma simbólica como numérica.

Luego, se define una función que implementa el método de Euler modificado. Este método es una mejora del método de Euler clásico para resolver ecuaciones diferenciales. Funciona calculando un valor predictivo inicial y luego corrigiéndolo usando un promedio de pendientes, lo cual mejora la precisión del resultado. La función también evalúa la solución exacta (si está disponible), compara ambos resultados y muestra el error cometido en cada paso.

Después de definir la función, el código la aplica a cuatro ejercicios distintos,

cada uno con su propia ecuación diferencial, solución exacta, condiciones iniciales y parámetros de integración como el tamaño del paso. Para cada ejercicio, se ejecuta el método numérico, se imprime una tabla con los resultados aproximados, reales y sus errores, y se genera una gráfica comparativa entre la solución numérica y la solución exacta.

Finalmente, se trabaja el ejercicio 15, donde se estudia un proceso químico complejo modelado por una ecuación diferencial sin solución exacta conocida. En este caso, solo se calcula y grafica la solución aproximada, mostrando la cantidad de producto formado en función del tiempo. También se imprime específicamente el valor inicial y el valor al final del intervalo para observar la variación.

Código 5.5.

Este código trabaja con la resolución numérica de ecuaciones diferenciales ordinarias utilizando principalmente el **método de Runge-Kutta-Fehlberg**, además del **método $solve_{ivp}$** de **SciPy** para otros casos.

Primero, se importan las bibliotecas necesarias: **NumPy** para cálculos numéricos, **Matplotlib** para graficar, **SymPy** para funciones simbólicas y **SciPy** para resolver ecuaciones diferenciales de manera más automatizada con **`solve_ivp`**.

Luego, se define una función personalizada que implementa el **método RKF45**, el cual es un método adaptativo de paso variable. Este método calcula varias pendientes intermedias **valores k1 a k6** que luego se combinan para obtener dos aproximaciones de la solución. La diferencia entre estas dos sirve para estimar el error y ajustar el tamaño del paso: si el error es aceptable, el paso se acepta y se avanza, y si no, se reduce el paso y se vuelve a intentar.

Se definen cuatro ejercicios simbólicos con soluciones exactas. Cada uno tiene una ecuación diferencial distinta, una condición inicial, y una función que representa la solución real. Estas ecuaciones se almacenan en un diccionario para ser recorridas en un ciclo.

Cada ejercicio es resuelto usando el método RKF45 personalizado. Se obtienen los tiempos, las soluciones aproximadas, y se calcula el error absoluto comparando con la solución exacta. Luego se imprime una tabla con estos resultados y se genera una gráfica para visualizar la comparación entre la solución numérica y la exacta.

Después de estos ejercicios, el código aborda otros problemas adicionales usando **`solve_ivp`**, que es un solucionador general de ecuaciones diferenciales ordinarias. Se define una función auxiliar que resuelve un inciso dado con este método. También compara con la solución exacta si está disponible, y calcula los errores. Cada uno de estos incisos representa un sistema diferente con condiciones y tolerancias propias.

En resumen, este código demuestra cómo usar dos enfoques distintos para resolver ecuaciones diferenciales: uno completamente personalizado con control de error **RKF45**, y otro utilizando una herramienta de biblioteca **`solve_ivp`**. Ambos permiten estudiar el comportamiento de soluciones aproximadas frente a soluciones exactas cuando se conocen.

Código 5.6.

Este código implementa la solución de ecuaciones diferenciales ordinarias utilizando el **método de Adams-Bashforth** de 4 pasos, iniciando el cálculo con el **método clásico de Runge-Kutta** de orden 4. Se usa una clase para encapsular la lógica y facilitar su reutilización con diferentes problemas.

Primero se definen las bibliotecas necesarias: NumPy para los cálculos numéricos y Matplotlib para las gráficas.

La clase principal, **AdamsBashforthSolver**, recibe la función de la ecuación diferencial, la condición inicial, los valores inicial y final de tiempo, el paso de integración, una función con la solución exacta opcional y un nombre para identificar el problema.

Dentro de la clase, el **método rk4_start** se encarga de obtener los tres primeros valores de la solución utilizando el método de Runge-Kutta de orden 4. Esto es necesario porque el método de Adams-Bashforth de 4 pasos necesita conocer los tres pasos anteriores para calcular el siguiente valor.

El **método solve** usa los valores iniciales calculados con RK4 y aplica luego el método de Adams-Bashforth. Este método es explícito y calcula la siguiente aproximación usando una combinación lineal de las derivadas en los cuatro puntos anteriores con coeficientes predeterminados.

También hay métodos auxiliares: **exact_values** calcula los valores de la solución exacta si está disponible, y **get_results** devuelve un diccionario con los tiempos, valores aproximados, exactos y los errores absolutos.

Fuera de la clase hay funciones para imprimir los resultados en forma tabular y para graficarlos. Luego, la función **solve_all_problems** define cuatro problemas diferentes con sus funciones derivadas y soluciones exactas. Cada uno es resuelto por un objeto de la clase **AdamsBashforthSolver**.

Finalmente, en la ejecución principal del programa, se resuelven todos los problemas definidos, se imprimen sus resultados y se generan sus respectivas gráficas comparando la solución numérica con la exacta.

Primer código de 5.9.

Este código simula el **modelo de Lotka-Volterra**, el cual describe la interacción entre una población de presas y una de depredadores mediante un sistema de ecuaciones diferenciales no lineales. Se definen cuatro parámetros: **k1** representa la tasa de natalidad de las presas, **k2** la tasa de mortalidad de presas debida a depredadores, **k3** la tasa de natalidad de los depredadores como resultado de la caza de presas, y **k4** la tasa de mortalidad natural de los depredadores. Las condiciones iniciales suponen 1000 presas y 500 depredadores. El sistema de ecuaciones describe el cambio en el tiempo de ambas poblaciones: la población de presas crece proporcionalmente a su tamaño y disminuye por interacción con los depredadores, mientras que la de depredadores aumenta por las mismas interacciones y decrece debido a la mortalidad natural.

El sistema se resuelve numéricamente utilizando el método de Runge-Kutta de orden 4(5), implementado con la función **solve_ivp** de SciPy, en el intervalo de

tiempo de 0 a 4 años, evaluado en 500 puntos uniformemente distribuidos. Los resultados se presentan en dos gráficas: una que muestra la evolución temporal de ambas poblaciones, y otra que representa el diagrama de fase, el cual permite visualizar los ciclos poblacionales.

También se calcula y se marca el punto de equilibrio, donde las derivadas son cero, es decir, donde las poblaciones no cambian si permanecen constantes. Finalmente, se imprime una interpretación cualitativa del comportamiento del sistema: las poblaciones tienden a oscilar de forma cíclica alrededor del punto de equilibrio, reflejando la dinámica natural de sistemas biológicos presa-depredador.

Segundo código de 5.9.

Este código implementa el **modelo de Lotka-Volterra** para estudiar la dinámica entre dos poblaciones: presas y depredadores. El sistema se basa en dos ecuaciones diferenciales que modelan cómo varían ambas poblaciones en el tiempo. Se definen los parámetros del sistema: la tasa de natalidad de presas, la tasa a la que las presas mueren por interacción con los depredadores, la tasa de natalidad de depredadores como resultado de la caza, y la tasa de mortalidad natural de los depredadores. Las condiciones iniciales son 1000 presas y 500 depredadores. La función `lotka_volterra` describe el sistema de ecuaciones diferenciales. La población de presas crece proporcionalmente a su cantidad y decrece debido al contacto con depredadores. La población de depredadores crece por el mismo contacto y decrece de forma natural.

Para resolver el sistema se utiliza el método de Runge-Kutta de orden 4(5), aplicado mediante `solve_ivp` de SciPy. La solución se evalúa en 500 puntos entre los años 0 y 4.

El resultado se representa gráficamente en dos formas. La primera muestra cómo evolucionan ambas poblaciones a lo largo del tiempo. La segunda es un diagrama de fase que grafica la población de depredadores contra la de presas, permitiendo observar ciclos de comportamiento.

Además, se calcula el punto de equilibrio del sistema, que es donde las tasas de cambio de ambas poblaciones son cero. Este punto se representa en el diagrama de fase.

Finalmente, se imprime una interpretación cualitativa: el modelo predice oscilaciones cíclicas entre presas y depredadores, donde un aumento de presas favorece a los depredadores, pero estos a su vez reducen la cantidad de presas, lo cual provoca una disminución en la población de depredadores, permitiendo que las presas se recuperen. Estas oscilaciones continúan indefinidamente según este modelo idealizado.

0.3 Conclusión

La implementación de múltiples métodos de resolución me permitió comparar su precisión, estabilidad y aplicabilidad en diferentes contextos. Los resultados obtenidos muestran cómo las aproximaciones numéricas, aunque susceptibles

al error, son herramientas poderosas cuando las soluciones exactas no están disponibles. Además, el uso de visualizaciones y análisis de errores me fue clave para lograr interpretar y validar los modelos desarrollados. En conjunto, esta tarea me reforzó tanto los fundamentos teóricos como las habilidades prácticas en la resolución computacional de ecuaciones diferenciales.

0.4 Resultados

Ejercicios 5.1.

```

C:\Users\PITEL\PycharmProjects\Practice 9\venv\Scripts\python.exe" "C:\Users\PITEL\PycharmProjects\Practice 9\venv\Scripts\python.exe"
Ejercicios 5.1
Ejercicio 1
Resultado= Eq(y(t), exp(sin(t)))
Resultado= Eq(y(t), t**2*(2*Ei(t) - 2*Ei(1)))
Resultado= Eq(y(t), 2*t*exp(t) - 4*exp(t) + 12*exp(t)/t - 12*exp(t)/t**2 + (sqrt(2)*E + 4*t)/t**2)
Resultado= Eq(y(t), exp(3*t)/(4*t + 1)**(5/4))
Inciso (a): 0f/0y = cos(t) (continua: SI)
Inciso (b): 0f/0y = 2/t (continua: SI)
Inciso (c): 0f/0y = -2/t (continua: SI)
Inciso (d): 0f/0y = 12*t/(4*t + 1) (continua: SI)

```

Figure 1: Eje. 1

```

Ejercicio 7
y1(t) usando Picard: t**2/2 + 1
Inciso a y1(t) usando Picard: t**2/2 + 1
Resultados del inciso b:
y0(t) = 1
y1(t) = t**2/2 + 1
y2(t) = -t**3/6 + t**2/2 + 1
y3(t) = t**4/24 - t**3/6 + t**2/2 + 1

```

Figure 2: Eje. 7

Ejercicios 5.2.

```
Ejercicio 1
Inciso a
Valores de inicio:
x = 0.00, y = 0.00
Iteración 1
x = 0.5000, y = 0.0000
Iteración 2
x = 1.0000, y = 1.1204
Inciso b
Valores de inicio:
x = 2.00, y = 1.00
Iteración 1
x = 2.5000, y = 2.0000
Iteración 2
x = 3.0000, y = 3.6250
Inciso c
Valores de inicio:
x = 1.00, y = 2.00
Iteración 1
x = 1.2500, y = 2.7500
Iteración 2
x = 1.5000, y = 3.5500
Iteración 3
x = 1.7500, y = 4.3917
Iteración 4
x = 2.0000, y = 5.2690
Inciso d
Valores de inicio:
x = 0.00, y = 1.00
Iteración 1
x = 0.2500, y = 1.2500
Iteración 2
```

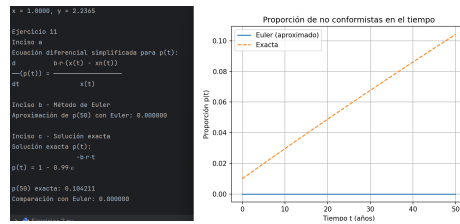


Figure 4: Eje. 11

Ejercicios 5.3.

```

C:\Users\PITEL\PycharmProjects\Practica 9\.venv\Scripts\python.exe
Ejercicio 5.3 - Método de Taylor de segundo orden

Inciso a
Valores de inicio:
t = 0.00, y = 0.0000
Iteración 1: t = 0.5000, y = 0.1250
Iteración 2: t = 1.0000, y = 2.0232

Inciso b
Valores de inicio:
t = 2.00, y = 1.0000
Iteración 1: t = 2.5000, y = 1.7500
Iteración 2: t = 3.0000, y = 2.4258

Inciso c
Valores de inicio:
t = 1.00, y = 2.0000
Iteración 1: t = 1.2500, y = 2.7812
Iteración 2: t = 1.5000, y = 3.6125
Iteración 3: t = 1.7500, y = 4.4854
Iteración 4: t = 2.0000, y = 5.3940

Inciso d
Valores de inicio:
t = 0.00, y = 1.0000
Iteración 1: t = 0.2500, y = 1.3438
Iteración 2: t = 0.5000, y = 1.7722
Iteración 3: t = 0.7500, y = 2.1107
Iteración 4: t = 1.0000, y = 2.2016

```

Figure 5: Eje. 1

```

Ejercicio 7 - Movimiento de un proyectil con resistencia del aire

Inciso a) Velocidad desde t = 0.1 hasta t = 1.0:
t = 0.1 s, v = 6.9836 m/s
t = 0.2 s, v = 5.8370 m/s
t = 0.3 s, v = 4.7950 m/s
t = 0.4 s, v = 3.7732 m/s
t = 0.5 s, v = 2.7673 m/s
t = 0.6 s, v = 1.7734 m/s
t = 0.7 s, v = 0.7877 m/s
t = 0.8 s, v = -0.1934 m/s
t = 0.9 s, v = -1.1734 m/s
t = 1.0 s, v = -2.1509 m/s

Inciso b) Tiempo en que el proyectil alcanza su altura máxima:
El proyectil alcanza su altura máxima aproximadamente a los 0.8 segundos.

```

Figure 6: Eje. 7

Ejercicios 5.4.

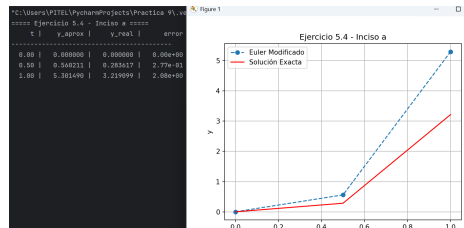


Figure 7: Eje. 1, Inciso a

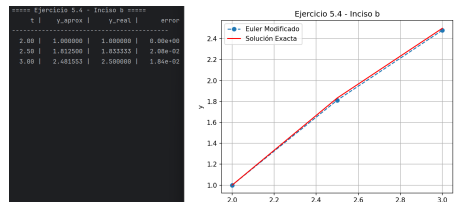


Figure 8: Eje. 1, Inciso b

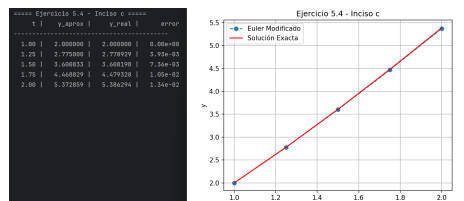


Figure 9: Eje. 1, Inciso c

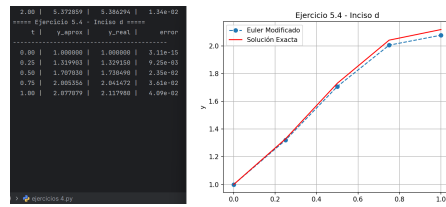


Figure 10: Eje. 1, Inciso d

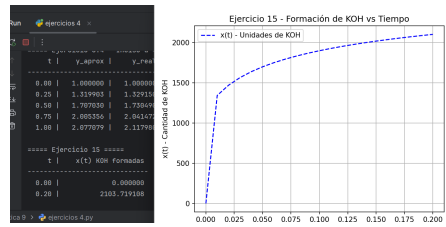


Figure 11: Eje. 15

Ejercicios 5.5.

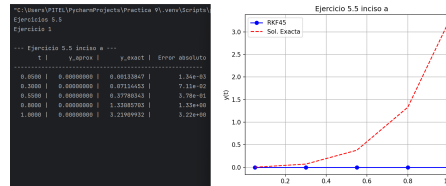


Figure 12: Eje. 1, Inciso a

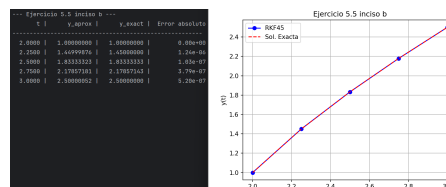


Figure 13: Eje. 1, Inciso b

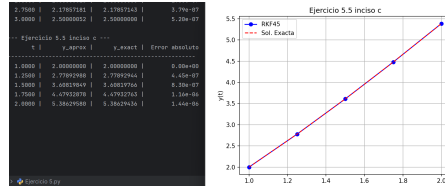


Figure 14: Eje. 1, Inciso c

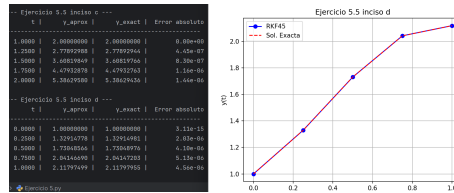


Figure 15: Eje. 1, Inciso d

```
t = 1.00000, y ≈ 1.00000
t = 1.00500, y ≈ 1.01004
t = 1.01000, y ≈ 1.02015
t = 1.01500, y ≈ 1.03034
t = 1.02000, y ≈ 1.04061
t = 1.02500, y ≈ 1.05095
t = 1.03000, y ≈ 1.06137
t = 1.03500, y ≈ 1.07187
t = 1.04000, y ≈ 1.08245
t = 1.04500, y ≈ 1.09312
t = 1.05000, y ≈ 1.10386
t = 1.05500, y ≈ 1.11468
t = 1.06000, y ≈ 1.12559
t = 1.06500, y ≈ 1.13658
t = 1.07000, y ≈ 1.14765
t = 1.07500, y ≈ 1.15881
t = 1.08000, y ≈ 1.17005
t = 1.08500, y ≈ 1.18138
t = 1.09000, y ≈ 1.19279
t = 1.09500, y ≈ 1.20430
t = 1.10000, y ≈ 1.21589
t = 1.10500, y ≈ 1.22757
t = 1.11000, y ≈ 1.23934
t = 1.11500, y ≈ 1.25120
t = 1.12000, y ≈ 1.26315
t = 1.12500, y ≈ 1.27520
t = 1.13000, y ≈ 1.28734
t = 1.13500, y ≈ 1.29957
t = 1.14000, y ≈ 1.31190
t = 1.14500, y ≈ 1.32432
t = 1.15000, y ≈ 1.33684
t = 1.15500, y ≈ 1.34946
t = 1.16000, y ≈ 1.36217
t = 1.16500, y ≈ 1.37499
t = 1.17000, y ≈ 1.38791
t = 1.17500, y ≈ 1.40092
t = 1.18000, y ≈ 1.41404
```

Figure 16: Eje. 2, parte 1

```
t = 1.09000, y ≈ 1.19279
t = 1.09500, y ≈ 1.20430
t = 1.10000, y ≈ 1.21589
t = 1.10500, y ≈ 1.22757
t = 1.11000, y ≈ 1.23934
t = 1.11500, y ≈ 1.25120
t = 1.12000, y ≈ 1.26315
t = 1.12500, y ≈ 1.27520
t = 1.13000, y ≈ 1.28734
t = 1.13500, y ≈ 1.29957
t = 1.14000, y ≈ 1.31190
t = 1.14500, y ≈ 1.32432
t = 1.15000, y ≈ 1.33684
t = 1.15500, y ≈ 1.34946
t = 1.16000, y ≈ 1.36217
t = 1.16500, y ≈ 1.37499
t = 1.17000, y ≈ 1.38791
t = 1.17500, y ≈ 1.40092
t = 1.18000, y ≈ 1.41404
t = 1.18500, y ≈ 1.42727
t = 1.19000, y ≈ 1.44060
t = 1.19500, y ≈ 1.45403
t = 1.20000, y ≈ 1.46757
t = 1.20000, y ≈ 1.46757
```

Figure 17: Eje. 2, parte 2

```

t = 2.00000, y ≈ 1.00000
t = 2.02500, y ≈ 1.04939
t = 2.05000, y ≈ 1.09762
t = 2.07500, y ≈ 1.14477
t = 2.10000, y ≈ 1.19091
t = 2.12500, y ≈ 1.23611
t = 2.15000, y ≈ 1.28043
t = 2.17500, y ≈ 1.32394
t = 2.20000, y ≈ 1.36667
t = 2.22500, y ≈ 1.40867
t = 2.25000, y ≈ 1.45000
t = 2.27500, y ≈ 1.49069
t = 2.30000, y ≈ 1.53077
t = 2.32500, y ≈ 1.57028
t = 2.35000, y ≈ 1.60926
t = 2.37500, y ≈ 1.64773
t = 2.40000, y ≈ 1.68571
t = 2.42500, y ≈ 1.72325
t = 2.45000, y ≈ 1.76034
t = 2.47500, y ≈ 1.79703
t = 2.50000, y ≈ 1.83333
t = 2.52500, y ≈ 1.86926
t = 2.55000, y ≈ 1.90484
t = 2.57500, y ≈ 1.94008
t = 2.60000, y ≈ 1.97500
t = 2.62500, y ≈ 2.00962
t = 2.65000, y ≈ 2.04394
t = 2.67500, y ≈ 2.07799
t = 2.70000, y ≈ 2.11176
t = 2.72500, y ≈ 2.14529
t = 2.75000, y ≈ 2.17857
t = 2.77500, y ≈ 2.21162
t = 2.80000, y ≈ 2.24444
t = 2.82500, y ≈ 2.27705
t = 2.85000, y ≈ 2.30946
t = 2.87500, y ≈ 2.34167
t = 2.90000, y ≈ 2.37368

```

Figure 18: Eje. 3, Inciso a, parte 1

```

t = 2.90000, y ≈ 2.37368
t = 2.92500, y ≈ 2.40552
t = 2.95000, y ≈ 2.43718
t = 2.97500, y ≈ 2.46867
t = 3.00000, y ≈ 2.50000
t = 3.00000, y ≈ 2.50000

```

Figure 19: Eje. 3, Inciso a, parte 2


```

Inciso 3b:
t = 1.00000, y ≈ 2.00000
t = 1.02500, y ≈ 2.07531
t = 1.05000, y ≈ 2.15123
t = 1.07500, y ≈ 2.22774
t = 1.10000, y ≈ 2.30484
t = 1.12500, y ≈ 2.38251
t = 1.15000, y ≈ 2.46073
t = 1.17500, y ≈ 2.53949
t = 1.20000, y ≈ 2.61879
t = 1.22500, y ≈ 2.69860
t = 1.25000, y ≈ 2.77893
t = 1.27500, y ≈ 2.85976
t = 1.30000, y ≈ 2.94107
t = 1.32500, y ≈ 3.02287
t = 1.35000, y ≈ 3.10514
t = 1.37500, y ≈ 3.18787
t = 1.40000, y ≈ 3.27106
t = 1.42500, y ≈ 3.35469
t = 1.45000, y ≈ 3.43877
t = 1.47500, y ≈ 3.52327
t = 1.50000, y ≈ 3.60820
t = 1.52500, y ≈ 3.69354
t = 1.55000, y ≈ 3.77930
t = 1.57500, y ≈ 3.86545
t = 1.60000, y ≈ 3.95201
t = 1.62500, y ≈ 4.03895
t = 1.65000, y ≈ 4.12628
t = 1.67500, y ≈ 4.21399
t = 1.70000, y ≈ 4.30207
t = 1.72500, y ≈ 4.39052
t = 1.75000, y ≈ 4.47933
t = 1.77500, y ≈ 4.56850
t = 1.80000, y ≈ 4.65802
t = 1.82500, y ≈ 4.74788
t = 1.85000, y ≈ 4.83809
t = 1.87500, y ≈ 4.92864

```

Figure 20: Eje. 3, Inciso b, parte 1

```

t = 1.87500, y ≈ 4.92864
t = 1.90000, y ≈ 5.01952
t = 1.92500, y ≈ 5.11073
t = 1.95000, y ≈ 5.20227
t = 1.97500, y ≈ 5.29412
t = 2.00000, y ≈ 5.38629
t = 2.00000, y ≈ 5.38629

```

Figure 21: Eje. 3, Inciso b, parte 2

Ejercicios 5.6.

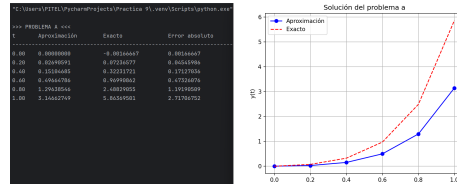


Figure 22: Eje. 1, Inciso a

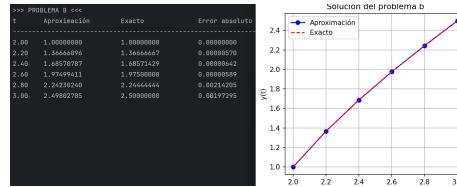


Figure 23: Eje. 1, Inciso b

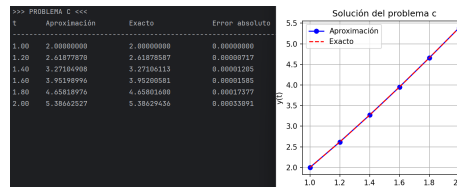


Figure 24: Eje. 1, Inciso c

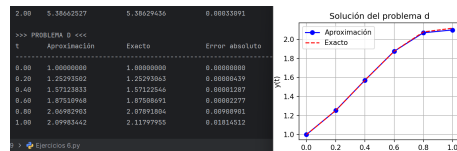


Figure 25: Eje. 1, Inciso d

Ejercicios 5.9.

```

C:\Users\PITEL\PycharmProjects\Practica 9\venv\Scripts\python.exe" "C:\Users\PITEL\PycharmProjects\Prac
Análisis de Estabilidad:
Punto de equilibrio:  $x_1 = 833.33$ ,  $x_2 = 1500.00$ 

Interpretación física:
1. Las poblaciones oscilan en ciclos alrededor del punto de equilibrio.
2. Cuando hay muchas presas, los depredadores aumentan, lo que luego reduce las presas.
3. Al disminuir las presas, los depredadores también disminuyen, permitiendo que las presas se recuperen.
4. Este ciclo se repite indefinidamente en el modelo clásico.

```

Figure 26: Eje. 1

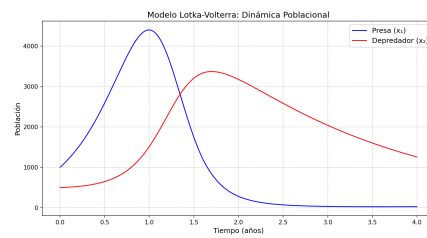


Figure 27: Eje. 1, Grafica 1

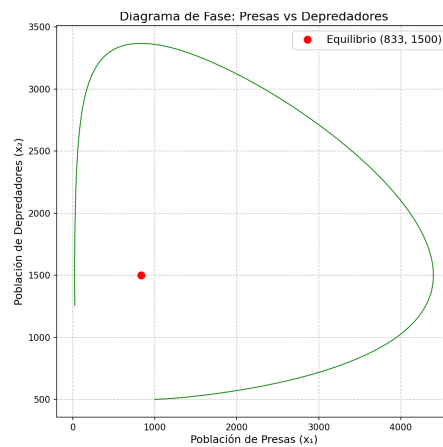


Figure 28: Eje. 1, Grafica 2

```

C:\Users\PITEL\PycharmProjects\Practica 9\venv\Scripts\python.exe" "C:\Users\PITEL\PycharmProjects\Prac
Análisis de Estabilidad:
Punto de equilibrio:  $x_1 = 833.33$ ,  $x_2 = 1500.00$ 

Interpretación física:
1. Las poblaciones oscilan en ciclos alrededor del punto de equilibrio.
2. Cuando hay muchas presas, los depredadores aumentan, lo que luego reduce las presas.
3. Al disminuir las presas, los depredadores también disminuyen, permitiendo que las presas se recuperen.
4. Este ciclo se repite indefinidamente en el modelo clásico.

```

Figure 29: Eje. 7

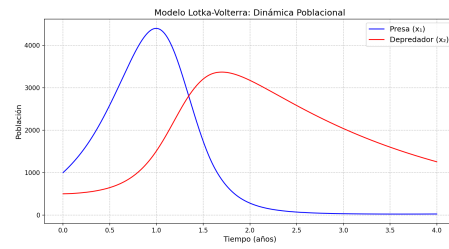


Figure 30: Eje. 7, Grafica 1

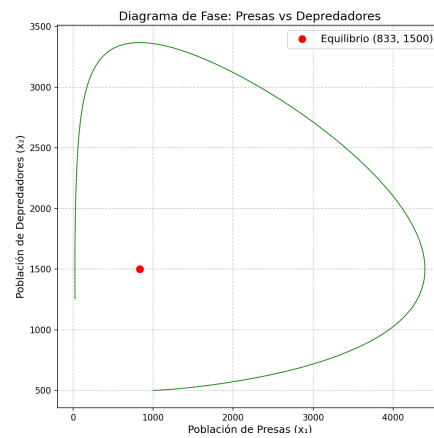


Figure 31: Eje. 7, Grafica 2

0.5 Anexos

Código de 5.1.

```
import sympy as sp
#Declarar variables simbólicas
t = sp.symbols('t')
y = sp.Function('y')
print("\nEjercicio 5.1\n Ejercicio 1")
#Inciso a
f_a = y(t)*sp.cos(t)
r_a = sp.solve(sp.Eq(sp.Derivative(y(t), "variables"), f_a), y(t), ics={y(0): 1})
print("Resultado a", r_a)
#Inciso b
f_b = (2/t) + y(t) + t**2 + sp.exp(t)
r_b = sp.solve(sp.Eq(sp.Derivative(y(t), "variables"), f_b), y(t), ics={y(1): 4})
print("Resultado b", r_b)
#Inciso c
f_c = -(2/t) * y(t) + t**2 + sp.exp(t)
r_c = sp.solve(sp.Eq(sp.Derivative(y(t), "variables"), f_c), y(t), ics={y(1): sp.sqrt(2)+sp.exp(1)})
print("Resultado c", r_c)
#Inciso d
f_d = (t**2+y(t))/(1+t**2)
r_d = sp.solve(sp.Eq(sp.Derivative(y(t), "variables"), f_d), y(t), ics={y(0): 1})
print("Resultado d", r_d)

#Verificar continuidad y condiciones de Lipschitz
for label, expr in zip(['a', 'b', 'c', 'd'], [f_a, f_b, f_c, f_d]):
    dr_dy = sp.diff(expr, "y")
    print(f"Inciso {label}: dr/dy = {dr_dy} (continua: Si)")
    t_tau = sp.symbols('tau')
    y = sp.Function('y')
    print("\nEjercicio 7")
    y0 = 1 # y(t) es constante
    f_tau = -y0 + tau + 1
    y1 = 1 + sp.integrate(f_tau, (tau, 0, t))
    print(f"y1(t) usando Picard:", y1)
#Ejercicio 7
#Declarar símbolos
t_tau = sp.symbols('tau')
y = sp.Function('y')
# Inciso a
y0 = 1
f_tau = -y0 + tau + 1
y1 = 1 + sp.integrate(f_tau, (tau, 0, t))
```

Figure 32: parte 1

```
print("Inciso a y1(t) usando Picard:", y1)
# Inciso b
yk = [sp.Integer(1)] # y0(t) = 1

for i in range(3): # Generar y1, y2, y3
    prev_y = yk[-1].subs(t, tau) # y_{k-1}(tau)
    f_tau = -prev_y + tau + 1
    y_next = 1 + sp.integrate(f_tau, (tau, 0, t))
    yk.append(sp.simplify(y_next))

# Mostrar resultados
print("Resultados del inciso b:")
for i, yi in enumerate(yk):
    print(f"y{i}(t) = {yi}")
```

Figure 33: parte 2

Código de 5.2.

```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt

print("Ejercicios 5.2")

print("\nEjercicio 1")
print("Inciso a")
x0 = 0.0
y0 = 0.0
xn = 1.0
h = 0.5
N = int((xn - x0) / h)
t = x0
y = y0
print("Valores de inicio:")
print(f"x = {t:.2f}, y = {y:.2f}")
for i in range(1, N + 1):
    y_a = t*sp.exp(3*t)-2*y
    y = y + h * y_a
    t = t + h
    print(f"Iteración {i}")
    print(f"x = {t:.4f}, y = {y:.4f}")

print("Inciso b")
x0 = 2.0
y0 = 1.0
xn = 3.0
h = 0.5
N = int((xn - x0) / h)
t = x0
y = y0
print("Valores de inicio:")
print(f"x = {t:.2f}, y = {y:.2f}")
for i in range(1, N + 1):
    y_b = 1 + (t-1)**2
    y = y + h * y_b
    t = t + h
    print(f"Iteración {i}")
```

Figure 34: parte 1

```

    print(f"x = {t:.4f}, y = {y:.4f}")

print("Inciso c")
x0 = 1.0
y0 = 2.0
xn = 2.0
h = 0.25
N = int((xn - x0) / h)
t = x0
y = y0
print("Valores de inicio:")
print(f"x = {t:.2f}, y = {y:.2f}")
for i in range(1, N + 1):
    y_a = 1 + y/t
    y = y + h * y_a
    t = t + h
    print(f"Iteración {i}")
    print(f"x = {t:.4f}, y = {y:.4f}")

print("Inciso d")
x0 = 0.0
y0 = 1.0
xn = 1.0
h = 0.25
N = int((xn - x0) / h)
t = x0
y = y0
print("Valores de inicio:")
print(f"x = {t:.2f}, y = {y:.2f}")
for i in range(1, N + 1):
    y_a = sp.cos(2*t) + sp.sin(3*t)
    y = y + h * y_a
    t = t + h
    print(f"Iteración {i}")
    print(f"x = {t:.4f}, y = {y:.4f}")

print("\nEjercicio 11")
print("Inciso a")

```

Figure 35: parte 2

```

t = sp.Symbol('t', real=True)
x = sp.Function('x')(t)
xn = sp.Function('xn')(t)
p = sp.Function('p')(t)
b, d, r = sp.symbols('b d r', positive=True)

# Ecuaciones dadas
dx_dt = sp.Eq(x.diff(t), (b - d)*x)
dxn_dt = sp.Eq(xn.diff(t), (b - d)*xn + r*b*(x - xn))
p_def = sp.Eq(p, xn / x)
dp_dt = sp.diff(p_def.rhs, 'symbolic:t')
dp_dt_simplified = sp.simplify(dp_dt.subs([(x.diff(t), dx_dt.rhs), (xn.diff(t), dxn_dt.rhs)]))
dp_dt_final = sp.Eq(dp_dt_simplified)

print("Ecuación diferencial simplificada para p(t):")
sp.pprint(dp_dt_final)

print("\nInicio b - Método de Euler")

# Parámetros dados
p0 = 0.01
b_val = 0.02
d_val = 0.015
r_val = 0.1
rb_val = r_val * b_val
T = 50
h = 1
n_steps = int(T / h)

# Método de Euler
t_values = np.arange(0, T + h, h)
p_values = np.zeros_like(t_values)
p_values[0] = p0

for i in range(n_steps):
    p_values[i+1] = p_values[i] + h * rb_val * (1 - p_values[i])

print("Aproximación de p(50) con Euler: {p_values[-1]:.6f}")

```

Figure 36: parte 3

```

print("Aproximación de p(50) con Euler: {p_values[-1]:.6f}")

print("\nInicio c - Solución exacta")

# Solución directa: p(t) = 1 - C*exp(-r*b*t)
C = sp.Symbol('C')
p_general = 1 - C * sp.exp(-r * b * t)
C_val = sp.solve(p_general.subs(t, sp.symbols('t')[0]) - p0, 'symbolic:C')[0]
p_solution = p_general.subs(C, C_val)
p_50 = p_solution.subs(t: 50, b: b_val, r: r_val)
p_50_val = float(p_50.evalf())

print("Solución exacta p(t):")
sp.pprint(sp.Eq(p, p_solution))
print("\np(50) exacta: {p_50_val:.6f}")
print("Comparación con Euler: {p_values[-1]:.6f}")

# Gráfica
plt.plot([t_val for t_val in t_values], p_values, label="Euler (aproximado)")
t_plot = np.linspace(start=0, stop=50, num=500)
p_exact_func = sp.lambdify(t, p_solution.subs({b: b_val, r: r_val}), modules="numpy")
plt.plot([t_val for t_val in t_plot], p_exact_func(t_plot), label="Exacta", linestyle='--')
plt.xlabel("Tiempo t (años)")
plt.ylabel("Proporción p(t)")
plt.title("Proporción de no conformistas en el tiempo")
plt.legend()
plt.grid(True)
plt.show()

```

Figure 37: parte 4

Código de 5.3.

```
import sympy as sp

print("Ejercicio 5.3 - Método de Taylor de segundo orden")

# Definir símbolos
t, y = sp.symbols('t y')

def metodo_taylor_orden_2(f_expr, y0, t0, h, n, inciso):
    f = sp.lambdify(args=(t, y), f_expr, modules='numpy')
    df = sp.diff(f_expr, symbols=t) + sp.diff(f_expr, symbols=y) * f_expr
    df_func = sp.lambdify(args=(t, y), df, modules='numpy')

    print(f"\nInciso {inciso}")
    print("Valores de inicio:")
    print(f"t = {t0:.2f}, y = {y0:.4f}")
    for i in range(1, n + 1):
        y1 = y0 + h * f(t0, y0) + (h**2 / 2) * df_func(t0, y0)
        t0 += h
        y0 = y1
        print(f"Iteración {i}: t = {t0:.4f}, y = {y0:.4f}")

# Inciso a: y' = t*e^(3t) - 2y, y(0)=0, h=0.5, 0 ≤ t ≤ 1
f_a = t * sp.exp(3*t) - 2*y
metodo_taylor_orden_2(f_a, y0=0, t0=0, h=0.5, n=2, inciso='a')

# Inciso b: y' = 1 + (t - y)^2, y(2)=1, h=0.5, 2 ≤ t ≤ 3
f_b = 1 + (t - y)**2
metodo_taylor_orden_2(f_b, y0=1, t0=2, h=0.5, n=2, inciso='b')

# Inciso c: y' = 1 + y/t, y(1)=2, h=0.25, 1 ≤ t ≤ 2
f_c = 1 + y/t
metodo_taylor_orden_2(f_c, y0=2, t0=1, h=0.25, n=4, inciso='c')

# Inciso d: y' = cos(2t) + sin(3t), y(0)=1, h=0.25, 0 ≤ t ≤ 1
f_d = sp.cos(2*t) + sp.sin(3*t)
metodo_taylor_orden_2(f_d, y0=1, t0=0, h=0.25, n=4, inciso='d')

# =====
# Ejercicio 7 - Proyectil con resistencia del aire
# =====
print("\nEjercicio 7 - Movimiento de un proyectil con resistencia del aire")
```

Figure 38: parte 1

```
t = sp.Symbol('t')
v = sp.Function('v')(t)

# Datos del problema
m = 0.11 # kg
g = 9.8 # m/s^2
h = 0.002 # kg/n
v0 = 0.0 # m/s

# Ecuación diferencial: m*v' = -mg - h*v*|v|
# Esto se transforma en: dv/dt = -g - (h/m)*v*|v|
def dvdt(v):
    return -g - (h/m)*v*abs(v)

# Método de Euler para calcular la velocidad
t0 = 0
v_t = v0
h = 0.1
n = 10 # de 0.1 a 1.0 s

print("\nInciso a) Velocidad desde t = 0.1 hasta t = 1.0:")
for i in range(1, n+1):
    v_t = v_t + h * dvdt(v_t)
    t0 += h
    print(f"t = {t0:.1f} s, v = {v_t:.4f} m/s")

# Para el inciso b), buscamos cuándo v = 0 (altura máxima)
print("\nInciso b) Tiempo en que el proyectil alcanza su altura máxima:")

t0 = 0
v_t = v0
h = 0.01
while v_t > 0:
    v_t = v_t + h * dvdt(v_t)
    t0 += h

tiempo_max = round(t0, 1)
print(f"El proyectil alcanza su altura máxima aproximadamente a los {tiempo_max:.1f} segundos.")
```

Figure 39: parte 2

Código de 5.4.

```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

def metodo_euler_modificado(f_sym, y_exact_expr, t0, y0, h, t_final, titulo): #funcion
    t = sp.symbols('t')
    f_numeric = sp.lambdify([t], f_sym, module='numpy')
    y_exact_func = sp.lambdify(t, y_exact_expr, module='numpy')

    ts = np.arange(t0, t_final + h, h)
    ys = np.zeros(len(ts))
    ys[0] = y0

    for i in range(1, len(ts)):
        t_i = ts[i - 1]
        y_i = ys[i - 1]
        y_predictor = y_i + h * f_numeric(t_i, y_i)
        y_corrector = y_i + h / 2 * (f_numeric(t_i, y_i) + f_numeric(t_i + h, y_predictor))
        ys[i] = y_corrector

    y_exact = y_exact_func(ts)

    print("===== titulo =====")
    print("t(t):>40 | { 'y_exact':>100 | { 'y_real':>100 | { 'error':>100}")
    print("-----")
    for i in range(len(ts)):
        print("t={:11.6f} | (y[i]:10.6f) | (y_exact[i]:10.6f) | (abs(y[i] - y_exact[i]):10.2e)".format(
            ts[i], ys[i], y_exact[i], abs(ys[i] - y_exact[i])))

    # Grafica
    plt.plot([t0, t_final], ts, ys, 'r-', label='Euler Modificado')
    plt.plot([t0, t_final], ts, y_exact, 'b-', label='Solucion Exacta')
    plt.xlabel('t')
    plt.ylabel('y')
    plt.title(titulo)
    plt.legend()
    plt.grid(True)
    plt.show()
```

Figure 40: parte 1

```
t, y = sp.symbols('t y')
f_a = t + sp.exp(1 + t) - 1 - 2 * y
y_real_a = (1 / 5) * t + sp.exp(3 + t) - (1 / 25) * sp.exp(1 + t) + (1 / 25) * sp.exp(-2 + t)
metodo_euler_modificado(f_a, y_real_a, t0=0, y0=0, h=0.5, t_final=1.0, titulo="Ejercicio 5.4 - inciso a")

# ===== Ejercicio b =====
f_b = 1 + (t - y) ** 2
y_real_b = t + 1 / (1 - t)
metodo_euler_modificado(f_b, y_real_b, t0=0, y0=1, h=0.5, t_final=3.0, titulo="Ejercicio 5.4 - inciso b")

# ===== Ejercicio c =====
f_c = 1 + y / t
y_real_c = t + sp.ln(t) + 2 + t
metodo_euler_modificado(f_c, y_real_c, t0=1, y0=2, h=0.25, t_final=2.0, titulo="Ejercicio 5.4 - inciso c")

# ===== Ejercicio d =====
f_d = sp.cos(2 + t) + sp.sin(3 + t)
y_real_d = (1 / 2) * sp.sin(2 + t) - (1 / 3) * sp.cos(3 + t) + (4 / 3)
metodo_euler_modificado(f_d, y_real_d, t0=0, y0=1, h=0.25, t_final=1.0, titulo="Ejercicio 5.4 - inciso d")

# ===== Ejercicio 15 =====
print("===== Ejercicio 15 =====")

# Parametros
t, x = sp.symbols('t x')
h = 0.25e-19
n1 = n2 = 263
n3 = 563

# EDO: dx/dt = f(t, x)
f15_expr = h * (n1 - x / 2) ** 2 + (n2 - x / 2) ** 2 + (n3 - 5 * x / 4) ** 3
f15_numeric = sp.lambdify([t, x], f15_expr, module='numpy')

# Condiciones iniciales y parametros numericos
t0 = 0
x0 = 0
t_final = 0.2
h = 0.01
```

Figure 41: parte 2

```

ts = np.arange(t0, t_final + h, h)
xs = np.zeros(len(ts))
xs[0] = x0

for i in range(1, len(ts)):
    t_i = ts[i - 1]
    x_i = xs[i - 1]
    x_predict = x_i + h * f15_numeric(t_i, x_i)
    x_correct = x_i + h / 2 * (f15_numeric(t_i, x_i) + f15_numeric(t_i + h, x_predict))
    xs[i] = x_correct

print(f'{"t":>6} | {"x(t) KDH formadas":>20}')
print('=' * 30)
for i in range(len(ts)):
    if abs(ts[i] - 0.2) < 1e-6 or ts[i] == 0.8:
        print(f'{"t":>6.2f} | {"xs[i]:>20.6f}')

# Gráfica
plt.plot('wgs ts, xs, 'b--', label='x(t) - Unidades de KDH')
plt.xlabel('t (s)')
plt.ylabel('x(t) - Cantidad de KDH')
plt.title('Ejercicio 15 - Formación de KDH vs Tiempo')
plt.grid(True)
plt.legend()
plt.show()

```

Figure 42: parte 3

Código de 5.5.

```

import numpy as np
import matplotlib.pyplot as plt
from sympy import symbols, exp, sin, cos, ln, lambdify
from scipy.integrate import solve_ivp

print('Ejercicio 5.5')
print('Ejercicio 1')

# Método de Runge-Kutta-Fehlberg personalizado
def rkf45(f, t0, y0, tf, tol, hmax, hmin):
    ts = [t0]
    ys = [y0]
    h = hmax
    while t0 < tf:
        if t0 + h > tf:
            h = tf - t0
        k1 = h * f(t0, y0)
        k2 = h * f(t0 + h/4, y0 + k1/4)
        k3 = h * f(t0 + 3*h/8, y0 + 3*k1/32 + 9*k2/32)
        k4 = h * f(t0 + 12*h/32, y0 + 1932*k1/2197 - 7200*k2/2197 + 7296*k3/2197)
        k5 = h * f(t0 + h, y0 + 439*k1/216 - 8*k2 + 3680*k3/513 - 845*k4/4104)
        k6 = h * f(t0 + h/2, y0 - 8*k1/27 + 2*k2 - 3544*k3/2565 + 1859*k4/4104 - 11*k5/40)

        y_next = y0 + 25*k1/216 + 148*k2/2565 + 2197*k4/4104 - k5/5
        z_next = y0 + 16*k1/135 + 6656*k3/12825 + 28561*k4/56430 - 9*k5/50 + 2*k6/55

        err = abs(y_next - z_next)
        if err < tol:
            t0 += h
            y0 = y_next
            ts.append(t0)
            ys.append(y0)

        q = 0.84 * (tol / err)**0.25 if err != 0 else 2
        h = max(hmin, min(hmax, q*h))
    return np.array(ts), np.array(ys)

# Obtener ejercicios simbólicos con soluciones exactas
def obtener_ejercicios():
    pass

```

Figure 43: parte 1


```

        print('t = %s, y = %s' % (t, y))

    if not_exact:
        t_sym = symbols('t')
        y_real_func = lamda(t_sym, sol_exact, **kwargs)(numpy)
        y_real_vals = y_real_func(sol.t)
        errors = np.abs(sol.y[0] - y_real_vals)
        print('Maximum error is %s' % max(errors))
        for t, y_real, y_sym in zip(sol.t, sol.y[0], y_real_vals, errors):
            print('t = %s, y_real = %s, y_sym = %s, error = %s' % (t, y_real, y_sym, error))

# Parameters specifications
N0 = 1e4
T01 = 1e4

# --- SUBCLASS 2 ---
resolver_init('lar', lamda t, y: (y/t)**2 + y/t, Upper(1, 1.2), y0=1, N0=N0, h=0.05, T0=T01)

# --- SUBCLASS 3 ---
resolver_init('lar', lamda t, y: 1 + (t - y)**2, Upper(2, 3), y0=1, N0=N0, h=0.025, T0=T01)
resolver_init('lar', lamda t, y: 1 + y/t if t > 0 else 0, Upper(1, 3), y0=1, N0=N0, h=0.025, T0=T01)

```

Figure 46: parte 4

Código de 5.6.

```

import numpy as np
import matplotlib.pyplot as plt

class AdamsBashforthSolver:
    def __init__(self, f, y0, t0, tf, h, exact_solution=None, problem_name=""):
        self.f = f
        self.y0 = y0
        self.t0 = t0
        self.tf = tf
        self.h = h
        self.exact_solution = exact_solution
        self.problem_name = problem_name
        self.t_values = np.arange(t0, tf + h / 2, h)
        self.n = len(self.t_values)

    def rk4_start(self):
        y = np.zeros(self.n)
        y[0] = self.y0
        for i in range(min(3, self.n - 1)):
            t = self.t_values[i]
            k1 = self.h * self.f(t, y[i])
            k2 = self.h * self.f(t + self.h / 2, y[i] + k1 / 2)
            k3 = self.h * self.f(t + self.h / 2, y[i] + k2 / 2)
            k4 = self.h * self.f(t + self.h, y[i] + k3)
            y[i + 1] = y[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
        return y

    def solve(self):
        y = self.rk4_start()
        if self.n >= 4:
            for i in range(3, self.n - 1):
                t = self.t_values[i]
                f0 = self.f(t, y[i])
                f1 = self.f(t - self.h, y[i - 1])
                f2 = self.f(t - 2 * self.h, y[i - 2])
                f3 = self.f(t - 3 * self.h, y[i - 3])
                y[i + 1] = y[i] + self.h * (55 * f0 - 59 * f1 + 37 * f2 - 9 * f3) / 24
        return y

    def exact_values(self):

```

Figure 47: parte 1

```

        if self.exact_solution is None:
            return None
        return np.array([self.exact_solution(t) for t in self.t_values])

def get_results(self): 4 usages
    y_approx = self.solve()
    y_exact = self.exact_values() if self.exact_solution else None
    errors = np.abs(y_approx - y_exact) if y_exact is not None else None
    return {
        'problem': self.problem_name,
        't_values': self.t_values,
        'approx': y_approx,
        'exact': y_exact,
        'errors': errors
    }

def print_result(res): 1usage
    print(f'\n>>> PROBLEMA {res["problem"].upper()} <<<')
    print(f'{t:<8}{Aproximación:<20}{Exacto:<20}{Error absoluto:<20}')
    print("-" * 68)
    for i in range(len(res["t_values"])):
        t = res["t_values"][i]
        approx = res["approx"][i]
        exact = res["exact"][i] if res["exact"] is not None else np.nan
        error = res["errors"][i] if res["errors"] is not None else np.nan
        print(f'{t:<8.2f}{approx:<20.8f}{exact:<20.8f}{error:<20.8f}')

def plot_result(res): 1usage
    plt.figure(figsize=(8, 4))
    plt.plot([res["t_values"], res["approx"], 'bo-', label='Aproximación')
    if res["exact"] is not None:
        plt.plot([res["t_values"], res["exact"], 'r--', label='Exacto')
    plt.title(f'Solución del problema {res["problem"]}')
    plt.xlabel('t')
    plt.ylabel('y(t)')
    plt.grid(True)
    plt.legend()
    plt.show()

```

Figure 48: parte 2

```

def solve_all_problems(): 1usage
    f_a = lambda t, y: t * np.exp(3 * t) - 2 * y
    exact_a = lambda t: (1 / 3) * t * np.exp(3 * t) - (1 / 24) * np.exp(3 * t) + (1 / 25) * np.exp(-2 * t)
    solver_a = AdamsBashforthSolver(f_a, y0=0, t0=0, tf=2, h=0.2,
                                    exact_solution=exact_a, problem_name="a")

    f_b = lambda t, y: 1 + (t - y) ** 2
    exact_b = lambda t: t + 1 / (1 - t)
    solver_b = AdamsBashforthSolver(f_b, y0=1, t0=0, tf=2, h=0.2,
                                    exact_solution=exact_b, problem_name="b")

    f_c = lambda t, y: 3 * y / t
    exact_c = lambda t: t * np.log(t) + 2 * t
    solver_c = AdamsBashforthSolver(f_c, y0=2, t0=1, tf=2, h=0.2,
                                    exact_solution=exact_c, problem_name="c")

    f_d = lambda t, y: np.cos(2 * t) * np.sin(3 * t)
    exact_d = lambda t: (1 / 3) * np.sin(2 * t) - (1 / 3) * np.cos(3 * t) + 4 / 3
    solver_d = AdamsBashforthSolver(f_d, y0=1, t0=0, tf=1, h=0.2,
                                    exact_solution=exact_d, problem_name="d")

    return [
        solver_a.get_results(),
        solver_b.get_results(),
        solver_c.get_results(),
        solver_d.get_results()
    ]

if __name__ == "__main__":
    all_results = solve_all_problems()

    for result in all_results:
        print_result(result)
        plt_result(result)

```

Figure 49: parte 3

Código de 5.9. para el ejercicio 1

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Parámetros del modelo
k1 = 3.0 # Tasa de natalidad de presas
k2 = 0.002 # Tasa de mortalidad de presas por depredador
k3 = 0.0006 # Tasa de natalidad de depredadores por presa
k4 = 0.5 # Tasa de mortalidad de depredadores

# Condiciones iniciales
x1_0 = 1000 # Población inicial de presas
x2_0 = 500 # Población inicial de depredadores

# Sistema de ecuaciones diferenciales
def lotka_volterra(t, y):
    x1, x2 = y
    dx1dt = k1 * x1 - k2 * x1 * x2
    dx2dt = k3 * x1 * x2 - k4 * x2
    return [dx1dt, dx2dt]

# Intervalo de tiempo
t_span = (0, 4)
t_eval = np.linspace(start=0, stop=4, num=500)

# Resolver el sistema
sol = solve_ivp(lotka_volterra, t_span, y0=[x1_0, x2_0], t_eval=t_eval, method='RK45')

# Extraer resultados
t = sol.t
x1 = sol.y[0]
x2 = sol.y[1]

# Gráfica de las poblaciones en el tiempo
plt.figure(figsize=(12, 8))
plt.plot('m', t, x1, 'b-', label='Presa (x1)')
plt.plot('m', t, x2, 'r-', label='Depredador (x2)')
plt.title('Modelo Lotka-Volterra: Dinámica Poblacional', fontsize=14)
plt.xlabel('Tiempo (años)', fontsize=12)
plt.ylabel('Población', fontsize=12)
plt.legend(fontsize=12)
```

Figure 50: parte 1

```
plt.grid(color='red', linestyle='--', alpha=0.7)
plt.show()

# Gráfica de fase (x1 vs x2)
plt.figure(figsize=(8, 8))
plt.plot('m', x1, x2, 'b-', linestyle=1)
plt.title('Diagrama de Fase: Presas vs Depredadores', fontsize=14)
plt.xlabel('Población de presas (x1)', fontsize=12)
plt.ylabel('Población de depredadores (x2)', fontsize=12)
plt.grid(color='red', linestyle='--', alpha=0.7)

# Punto de equilibrio estable
x1_eq = k4 / k3
x2_eq = k1 / k2
plt.plot([x1_eq, x2_eq], 'r-', marker='x', label='Equilibrio (x1_eq:0.7, x2_eq:0.7)')
plt.legend(fontsize=12)
plt.show()

# Análisis de estabilidad
print("Análisis de Estabilidad:")
print(f"Punto de equilibrio: x1 = {x1_eq:.2f}, x2 = {x2_eq:.2f}")
print("Evaluación de la estabilidad:")
print("1. Las poblaciones oscilan en ciclos alrededor del punto de equilibrio.")
print("2. Cuando hay muchas presas, los depredadores aumentan, lo que luego reduce las presas.")
print("3. Al disminuir las presas, los depredadores también disminuyen, permitiendo que las presas se recuperen.")
print("4. Este ciclo se repite constantemente en el mundo biológico."
```

Figure 51: parte 2

Código de 5.9. para el ejercicio 7

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# Parámetros del modelo
k1 = 3.0 # Tasa de natalidad de presas
k2 = 0.002 # Tasa de mortalidad de presas por depredador
k3 = 0.0086 # Tasa de natalidad de depredadores por presa
k4 = 0.5 # Tasa de mortalidad de depredadores

# Condiciones iniciales
x1_0 = 1000 # Población inicial de presas
x2_0 = 500 # Población inicial de depredadores

# Sistema de ecuaciones diferenciales
def lotka_volterra(t, y):
    x1, x2 = y
    dx1dt = k1 * x1 - k2 * x1 * x2
    dx2dt = k3 * x1 * x2 - k4 * x2
    return [dx1dt, dx2dt]

# Intervalo de tiempo
t_span = (0, 4)
t_eval = np.linspace(start=0, stop=4, num=500)

# Resolver el sistema
sol = solve_ivp(lotka_volterra, t_span, y0=[x1_0, x2_0], t_eval=t_eval, method='RK45')

# Extraer resultados
t = sol.t
x1 = sol.y[0]
x2 = sol.y[1]

# Gráfica de las poblaciones en el tiempo
plt.figure(figsize=(12, 6))
plt.plot('w', t, x1, 'b-', label='Presa (x1)')
plt.plot('w', t, x2, 'r-', label='Depredador (x2)')
plt.title('Modelo Lotka-Volterra: Dinámica Poblacional', fontsize=14)
plt.xlabel('Tiempo (años)', fontsize=12)
plt.ylabel('Población', fontsize=12)
plt.legend(fontsize=12)
```

Figure 52: parte 1

```
plt.grid(basis='log', linestyle='--', alpha=0.5)
plt.show()

# Gráfica de fase (x1 vs x2)
plt.figure(figsize=(8, 6))
plt.plot('w', x1, x2, 'g-', linestyle=1)
plt.title('Diagrama de Fase: Presas vs Depredadores', fontsize=14)
plt.xlabel('Población de Presas (x1)', fontsize=12)
plt.ylabel('Población de Depredadores (x2)', fontsize=12)
plt.grid(basis='log', linestyle='--', alpha=0.5)

# Punto de equilibrio estable
x1_eq = k4 / k3
x2_eq = k1 / k2
plt.plot('w', x1_eq, x2_eq, 'rs', markersize=4, label=f'Equilibrio ((x1_eq, x2_eq))')
plt.legend(fontsize=12)
plt.show()

# Análisis de estabilidad
print('Validación de estabilidad:')
print(f'Punto de equilibrio: x1 = {x1_eq}, x2 = {x2_eq}')
print(f'Interpretación física:')
print(f'1. Las poblaciones estables en ciclos alrededor del punto de equilibrio.')
print(f'2. Cuando hay muchas presas, los depredadores aumentan, lo que luego reduce las presas.')
print(f'3. Al disminuir las presas, los depredadores también disminuyen, permitiendo que las presas se recuperen.')
print(f'4. Este ciclo se repite periódicamente en el modelo biológico.')

```

Figure 53: parte 2